

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – DATA CLEANING CHALLENGE

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO3	Assessment:	Tool 2 – Data Cleaning Challenge
Weightage:		Total Marks:	
CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL5 and BL6).			
Student Name:	Dharshini H	Reg. No.:	192324268
Section / Batch:	B.Tech AI & DS	Date of Submission:	11-08-2026
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I Dharshini H 192324268 (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Dharshini H

Anomaly Detection (5 Marks)	Cleaning Methods (5 Marks)	Data Transformation (5 Marks)	Analytical Comparison (5 Marks)	Governance Discussion (5 Marks)	Total (25 Marks)

Question 1: E-Commerce Customer Transactions Dataset

An online retail company has collected customer transaction data from multiple sales channels. The management wants to identify the top-performing customer segments and products. However, the dataset contains several quality issues.

CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City
C101	Raj Kumar	Male	28	Electronics	2	15000 INR	10-01-2025	Chennai
C102	Priya S	Female	35	Fashion	3	1200 INR	11-01-2025	chennai
C103	Arun V	Male	150	Electronics	1	300 USD	12-01-2025	CHENNAI
C104	Kavya R	Female	0	Grocery	-2	800 INR	15-01-2025	Coimbatore
C105	Rahul P	Male	42	Fashion	5	2500 INR	18-01-2025	Madurai
C105	Rahul P	Male	42	Fashion	5	2500 INR	18-01-2025	Madurai
NULL	Sneha M	Female	30	Electronics	2	450 USD	05-02-2025	Trichy
C107	Deepak N	Male		Grocery	3	600 INR	07-02-2025	Tiruchirappalli
C108	Meena K	Female	27	Fashion	1	1800 INR	2025-13-01	Salem
C109	John D	Male	31	Electronics	4	250 EUR	15-02-2025	Chennai

Data Issues

- Missing Customer IDs
- Duplicate transactions
- Negative quantities
- Unit prices recorded in different currencies
- Age values such as 0, 150, and blanks
- Inconsistent city names (e.g., Chennai, chennai, CHENNAI)
- Incorrect purchase dates

Tasks

1. Identify and classify all data quality issues. (5 Marks)
2. Design a cleaning strategy for each issue. (5 Marks)
3. Perform data standardization and transformation. (5 Marks)
4. Analyze how cleaning affects total revenue calculations. (5 Marks)
5. Generate a report discussing business risks if the data were not cleaned. (5 Marks)

ANSWER:

1. Anomaly Detection

The following data quality anomalies are identified:

1. Missing values: CustomerID is missing for Sneha M and Age is missing for Deepak N.
2. Duplicate record: Customer C105's transaction appears twice with identical details.
3. Invalid values: Age 0 and 150 are unrealistic, and Quantity -2 is invalid for a normal purchase.
4. Currency inconsistency: Unit prices are recorded in INR, USD, and EUR.
5. Inconsistent/invalid formats: City names such as Chennai, chennai, and CHENNAI differ, and 2025-13-01 is an invalid date.

2. Cleaning Methods

Data Issue	Cleaning Method
Missing CustomerID	Recover from the customer database or assign a unique temporary ID
Duplicate C105	Identify duplicate rows and retain only one valid transaction
Negative quantity	Verify whether it represents a return; otherwise correct/remove it
Invalid ages	Replace with NULL and investigate or impute using valid customer information
Missing age	Impute using a suitable method such as median age
Different currencies	Convert all currencies into INR using the appropriate exchange rate
Inconsistent city names	Standardize all city names, e.g., chennai → Chennai
Invalid date	Verify against the source; otherwise mark as missing
Date format	Convert all valid dates to YYYY-MM-DD

3. Data Transformation

The following transformations are performed:

1. Currency transformation: Separate the numeric price and currency, then convert USD and EUR prices to INR.
2. Date transformation: Convert dates such as 10-01-2025 into the standard format 2025-01-10.
3. City standardization: Convert chennai and CHENNAI to Chennai.
4. Age transformation: Treat 0, 150, and blank values as invalid/missing and replace or impute them after verification.
5. Duplicate and invalid transaction transformation: Remove duplicate C105 and investigate the negative quantity transaction before calculating revenue.

4. Analytical Comparison

Before Cleaning

- Duplicate C105 transaction causes revenue to be overestimated by ₹12,500.
- Negative quantity for C104 produces negative revenue of ₹1,600.

- USD and EUR values cannot be directly added to INR.
- Chennai may be incorrectly treated as three separate city categories.
- Invalid dates and ages can produce incorrect time-based and customer-segment analysis.

After Cleaning

- Duplicate transactions are removed.
- Invalid quantities are corrected or treated as returns.
- All currencies are converted to INR.
- City names are standardized.
- Invalid/missing ages and dates are handled appropriately.
- Revenue and customer/product analysis become more accurate.

Therefore, cleaning changes the analytical results from unreliable raw statistics to consistent and trustworthy business insights.

5. Governance Discussion

Proper data governance should be implemented to prevent similar problems in the future.

1. Data validation rules: Set constraints for age, quantity, price and date during data entry.
2. Standardization: Define common formats for currency, city names, dates and customer IDs.
3. Duplicate prevention: Use unique transaction IDs to prevent the same transaction from being recorded multiple times.
4. Data quality monitoring: Regularly check completeness, validity, consistency and accuracy using automated data-quality reports.
5. Data ownership and security: Assign responsibility for maintaining customer and transaction data, control access, and maintain audit trails for corrections.

```
CO3 > at1_1.py > ...
1  import pandas as pd
2
3  # 1. CREATE THE DATASET
4
5  data = {
6      "CustomerID": ["C101", "C102", "C103", "C104", "C105", "C105", "None", "C107", "C108", "C109"],
7      "CustomerName": ["Raj Kumar", "Priya S", "Arun V", "Kavya R", "Rahul P", "Rahul P", "Sneha M", "Deepak N", "Meena K", "John D"],
8      "Gender": ["Male", "Female", "Male", "Female", "Male", "Male", "Female", "Male", "Female", "Male"],
9      "Age": [28, 35, 150, 0, 42, 42, 30, None, 27, 31],
10     "ProductCategory": ["Electronics", "Fashion", "Electronics", "Grocery", "Fashion", "Fashion", "Electronics", "Grocery", "Fashion",
11                          "Electronics"],
12     "Quantity": [2, 3, 1, -2, 5, 5, 2, 3, 1, 4],
13     "UnitPrice": ["15000 INR", "1200 INR", "300 USD", "800 INR", "2500 INR", "2500 INR", "450 USD", "600 INR", "1800 INR", "250 EUR"],
14     "PurchaseDate": ["10-01-2025", "11-01-2025", "12-01-2025", "15-01-2025", "18-01-2025", "18-01-2025", "05-02-2025", "07-02-2025",
15                     "2025-13-01", "15-02-2025"],
16     "City": ["Chennai", "chennai", "CHENNAI", "Coimbatore", "Madurai", "Madurai", "Trichy", "Tiruchirappalli", "Salem", "Chennai"]
17 }
18
19 df = pd.DataFrame(data)
20 print("===== ORIGINAL DATA =====")
21 print(df)
```

```

22  ..
23  # 2. ANOMALY DETECTION
24
25  print("\n===== ANOMALY DETECTION =====")
26  # Missing values
27  print("\nMissing Values:")
28  print(df.isnull().sum())
29  # Duplicate records
30  print("\nDuplicate Records:")
31  print(df[df.duplicated(keep=False)])
32  # Invalid ages
33  invalid_age = df[(df["Age"] <= 0) | (df["Age"] > 100)]
34  print("\nInvalid Ages:")
35  print(invalid_age[["CustomerID", "CustomerName", "Age"]])
36  # Missing ages
37  print("\nMissing Ages:")
38  print(df[df["Age"].isnull()][["CustomerID", "CustomerName", "Age"]])
39  # Negative quantities
40  print("\nNegative Quantities:")
41  print(df[df["Quantity"] < 0])
42  # Invalid dates
43  date_check = pd.to_datetime(
44      df["PurchaseDate"],
45      dayfirst=True,
46      errors="coerce"
47  )
48  print("\nInvalid Dates:")
49  print(df[date_check.isnull()][["CustomerID", "PurchaseDate"]])
50  # Currency detection
51  print("\nCurrency Values:")
52  print(df["UnitPrice"].str.extract(r"([A-Za-z]+)$")[0].value_counts())
53  # Inconsistent city names
54  print("\nCity Values:")
55  print(df["City"].unique())
56  ..

```

```

56
57  # 3. CLEANING METHODS
58
59  cleaned_df = df.copy()
60  # ----- Missing CustomerID -----
61  # Assign a unique ID for the missing customer
62  cleaned_df.loc[
63      cleaned_df["CustomerID"].isnull(),
64      "CustomerID"
65  ] = "C106"
66
67  # ----- Remove duplicate transactions -----
68  cleaned_df = cleaned_df.drop_duplicates()
69
70  # ----- Handle negative quantity -----
71  # Here we assume it is an invalid sales quantity.
72  # Replace it with missing value for investigation.
73  cleaned_df.loc[
74      cleaned_df["Quantity"] < 0,
75      "Quantity"
76  ] = None
77
78  # ----- Handle invalid ages -----
79  cleaned_df.loc[
80      (cleaned_df["Age"] <= 0) |
81      (cleaned_df["Age"] > 100),
82      "Age"
83  ] = None
84
85  # ----- Fill missing age using median -----
86  median_age = cleaned_df["Age"].median()
87
88  cleaned_df["Age"] = cleaned_df["Age"].fillna(median_age)
89
90  # ----- Standardize city names -----
91  cleaned_df["City"] = cleaned_df["City"].str.title()
92
93  # Standardize Trichy name
94  cleaned_df["City"] = cleaned_df["City"].replace({
95      "Trichy": "Tiruchirappalli"
96  })
97

```

```

99 # 4. DATA TRANSFORMATION
100
101 # Extract numeric price
102 cleaned_df["Price"] = cleaned_df["UnitPrice"].str.extract(
103     r"([\d.]+)"
104 ).astype(float)
105 # Extract currency
106 cleaned_df["Currency"] = cleaned_df["UnitPrice"].str.extract(
107     r"([A-Za-z]+)$"
108 )
109 # ----- Currency Conversion -----
110 # Example exchange rates
111 exchange_rates = {
112     "INR": 1,
113     "USD": 85,
114     "EUR": 90
115 }
116 cleaned_df["Price_INR"] = (
117     cleaned_df["Price"] *
118     cleaned_df["Currency"].map(exchange_rates)
119 )
120 # ----- Standardize dates -----
121 cleaned_df["PurchaseDate"] = pd.to_datetime(
122     cleaned_df["PurchaseDate"],
123     dayfirst=True,
124     errors="coerce"
125 )
126 # Convert to YYYY-MM-DD
127 cleaned_df["PurchaseDate"] = (
128     cleaned_df["PurchaseDate"].dt.strftime("%Y-%m-%d"))
129 # ----- Calculate Revenue -----
130 cleaned_df["Revenue_INR"] = (
131     cleaned_df["Quantity"] *
132     cleaned_df["Price_INR"])
133 print("\n===== CLEANED DATA =====")
134 print(cleaned_df)
135 # 5. ANALYTICAL COMPARISON
136 print("\n===== ANALYTICAL COMPARISON =====")
137 # Raw revenue
138 raw_temp = df.copy()
139 raw_temp["Price"] = raw_temp["UnitPrice"].str.extract(
140     r"([\d.]+)"
141 ).astype(float)
142 raw_temp["Currency"] = raw_temp["UnitPrice"].str.extract(
143     r"([A-Za-z]+)$"
144 )
145 raw_temp["Revenue"] = (
146     raw_temp["Quantity"] * raw_temp["Price"]
147 )
148 raw_revenue = raw_temp["Revenue"].sum()
149 # Cleaned revenue
150 cleaned_revenue = cleaned_df["Revenue_INR"].sum()
151 print("Raw arithmetic revenue:", raw_revenue)
152 print("Cleaned revenue in INR:", cleaned_revenue)
153 print("\nRevenue Difference:")
154 print(cleaned_revenue - raw_revenue)

```

```

155
156 # 6. PRODUCT CATEGORY ANALYSIS
157
158 print("\n===== PRODUCT PERFORMANCE =====")
159 product_sales = cleaned_df.groupby(
160     "ProductCategory"
161 )["Revenue_INR"].sum().sort_values(
162     ascending=False
163 )
164 print(product_sales)
165
166 # 7. CITY-WISE ANALYSIS
167 print("\n===== CITY-WISE REVENUE =====")
168 city_sales = cleaned_df.groupby(
169     "City"
170 )["Revenue_INR"].sum().sort_values(
171     ascending=False
172 )
173 print(city_sales)

```

```

...
175 # 8. GOVERNANCE REPORT
176
177 print("\n===== DATA GOVERNANCE REPORT =====")
178 print("""
179 1. Data validation rules should be applied during data entry.
180 2. CustomerID and TransactionID should be unique.
181 3. Age should be within a valid range.
182 4. Quantity should not be negative unless it represents a return.
183 5. All currencies should be converted to a common reporting currency.
184 6. Dates should follow YYYY-MM-DD format.
185 7. City names should follow a standard naming convention.
186 8. Missing values should be monitored regularly.
187 9. Duplicate transactions should be automatically detected.
188 10. Data corrections should be recorded using audit logs.
189 """)

```

===== ORIGINAL DATA =====

	CustomerID	CustomerName	Gender	Age	...	Quantity	UnitPrice	PurchaseDate	City
0	C101	Raj Kumar	Male	28.0	...	2	15000 INR	10-01-2025	Chennai
1	C102	Priya S	Female	35.0	...	3	1200 INR	11-01-2025	chennai
2	C103	Arun V	Male	150.0	...	1	300 USD	12-01-2025	CHENNAI
3	C104	Kavya R	Female	0.0	...	-2	800 INR	15-01-2025	Coimbatore
4	C105	Rahul P	Male	42.0	...	5	2500 INR	18-01-2025	Madurai
5	C105	Rahul P	Male	42.0	...	5	2500 INR	18-01-2025	Madurai
6	NaN	Sneha M	Female	30.0	...	2	450 USD	05-02-2025	Trichy
7	C107	Deepak N	Male	NaN	...	3	600 INR	07-02-2025	Tiruchirappalli
8	C108	Meena K	Female	27.0	...	1	1800 INR	2025-13-01	Salem
9	C109	John D	Male	31.0	...	4	250 EUR	15-02-2025	Chennai

[10 rows x 9 columns]

===== ANOMALY DETECTION =====

Missing Values:

```

CustomerID      1
CustomerName     0
Gender           0
Age              1
ProductCategory  0
Quantity         0
UnitPrice        0
PurchaseDate     0
City             0
dtype: int64

```

Duplicate Records:

	CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City
4	C105	Rahul P	Male	42.0	Fashion	5	2500 INR	18-01-2025	Madurai
5	C105	Rahul P	Male	42.0	Fashion	5	2500 INR	18-01-2025	Madurai

Invalid Ages:

	CustomerID	CustomerName	Age
2	C103	Arun V	150.0
3	C104	Kavya R	0.0

Missing Ages:

	CustomerID	CustomerName	Age
7	C107	Deepak N	NaN

Negative Quantities:

	CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City
3	C104	Kavya R	Female	0.0	Grocery	-2	800 INR	15-01-2025	Coimbatore

```

Invalid Dates:
  CustomerID PurchaseDate
8      C108    2025-13-01

Currency Values:
0
INR    7
USD    2
EUR    1
Name: count, dtype: int64

City Values:
<StringArray>
[      'Chennai',      'chennai',      'CHENNAI',      'Coimbatore',
      'Madurai',      'Trichy', 'Tiruchirappalli',      'Salem']
Length: 8, dtype: str

===== CLEANED DATA =====
  CustomerID CustomerName Gender  Age  ...  Price  Currency  Price_INR  Revenue_INR
0      C101    Raj Kumar   Male  28.0  ...  15000.0      INR    15000.0    30000.0
1      C102    Priya S   Female  35.0  ...   1200.0      INR     1200.0     3600.0
2      C103    Arun V   Male   30.5  ...    300.0      USD    25500.0    25500.0
3      C104    Kavya R   Female  30.5  ...    800.0      INR     800.0         NaN
4      C105    Rahul P   Male   42.0  ...   2500.0      INR    2500.0    12500.0
6      C106    Sneha M   Female  30.0  ...    450.0      USD    38250.0    76500.0
7      C107    Deepak N   Male   30.5  ...    600.0      INR     600.0     1800.0
8      C108    Meena K   Female  27.0  ...   1800.0      INR    1800.0     1800.0
9      C109    John D   Male   31.0  ...    250.0      EUR    22500.0    90000.0

[9 rows x 13 columns]

```

===== ANALYTICAL COMPARISON =====

Raw arithmetic revenue: 62800.0

Cleaned revenue in INR: 241700.0

Revenue Difference:

178900.0

===== PRODUCT PERFORMANCE =====

ProductCategory

Electronics 222000.0

Fashion 17900.0

Grocery 1800.0

Name: Revenue_INR, dtype: float64

===== CITY-WISE REVENUE =====

City

Chennai 149100.0

Tiruchirappalli 78300.0

Madurai 12500.0

Salem 1800.0

Coimbatore 0.0

Name: Revenue_INR, dtype: float64

===== DATA GOVERNANCE REPORT =====

1. Data validation rules should be applied during data entry.
2. CustomerID and TransactionID should be unique.
3. Age should be within a valid range.
4. Quantity should not be negative unless it represents a return.
5. All currencies should be converted to a common reporting currency.
6. Dates should follow YYYY-MM-DD format.
7. City names should follow a standard naming convention.
8. Missing values should be monitored regularly.
9. Duplicate transactions should be automatically detected.
10. Data corrections should be recorded using audit logs.

Question 2: Hospital Patient Records Dataset

A hospital wants to analyze patient admissions and treatment effectiveness. Data were entered manually by different departments, leading to inconsistencies.

PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID
P1001	Ravi Kumar	45	M	10-01-2025	15-01-2025	Diabetes	25000	D101
P1002	Priya S	38	F	12-01-2025	11-01-2025	Hypertension	18000	D102
P1003	Arun V	125	M	13-01-2025	20-01-2025	diabetes	22000	D101
NULL	Kavya R	29	F	15-01-2025	17-01-2025	Diabtes	15000	D103
P1005	Rahul P	51	M	18-01-2025	25-01-2025	Asthma	NULL	D104
P1005	Rahul P	51	M	18-01-2025	25-01-2025	Asthma	NULL	D104
P1006	Sneha M	34	F	15-01-2025	18-01-2025	Hypertension	12000	D102
P1007	Deepak N		M	01-02-2025	05-02-2025	Asthma	10000	D104
P1008	Meena K	26	F	10-02-2025	15-02-2025	HYPERTENSION	13500	D102
P1009	John D	40	M	12-02-2025	18-02-2025	Diabetes	20000	D101

Data Issues

- Missing patient IDs
- Duplicate patient records
- Discharge dates before admission dates
- Null treatment costs
- Inconsistent diagnosis names
- Multiple formats of dates

- Unrealistic patient ages (>120)

Tasks

1. Detect all inconsistencies and anomalies. (5 Marks)
2. Develop rules for handling missing and invalid values. (5 Marks)
3. Create a process to standardize diagnosis names. (5 Marks)
4. Evaluate how cleaning changes average treatment cost and hospital stay duration. (5 Marks)
5. Recommend data governance measures to prevent future errors. (5 Marks)

```
CO3 > at1_2.py > ...
1 import pandas as pd
2 # 1. CREATE DATASET
3 data = {
4     "PatientID": ["P1001", "P1002", "P1003", None, "P1005", "P1005", "P1006", "P1007", "P1008", "P1009"],
5     "Name": ["Ravi Kumar", "Priya S", "Arun V", "Kavya R", "Rahul P", "Rahul P", "Sneha M", "Deepak N", "Meena K", "John D"],
6     "Age": [45, 38, 125, 29, 51, 51, 34, None, 26, 40],
7     "Gender": ["M", "F", "M", "F", "M", "M", "F", "M", "F", "M"],
8     "AdmissionDate": ["10-01-2025", "12-01-2025", "13-01-2025", "15-01-2025", "18-01-2025", "18-01-2025", "15-01-2025", "01-02-2025", "10-02-2025", "12-02-2025"],
9     "DischargeDate": ["15-01-2025", "11-01-2025", "20-01-2025", "17-01-2025", "25-01-2025", "25-01-2025", "18-01-2025", "05-02-2025", "15-02-2025", "18-02-2025"],
10    "Diagnosis": ["Diabetes", "Hypertension", "diabetes", "Diabtes", "Asthma", "Asthma", "Hypertension", "Asthma", "HYPERTENSION", "Diabetes"],
11    "TreatmentCost": [25000, 18000, 22000, 15000, None, None, 12000, 10000, 13500, 20000],
12    "DoctorID": ["D101", "D102", "D101", "D103", "D104", "D104", "D102", "D104", "D102", "D101"]
13 }
14 df = pd.DataFrame(data)
15 print("=" * 70)
16 print("ORIGINAL HOSPITAL PATIENT DATA")
17 print("=" * 70)
18 print(df)
19
```

```
CO3 > at1_2.py > ...
22 # 2. ANOMALY DETECTION
23 print("\n" + "=" * 70)
24 print("1. ANOMALY DETECTION")
25 print("=" * 70)
26 # Missing values
27 print("\n--- Missing Values ---")
28 print(df.isnull().sum())
29 # Duplicate records
30 print("\n--- Duplicate Records ---")
31 duplicates = df[df.duplicated(keep=False)]
32 print(duplicates)
33 # Invalid ages
34 print("\n--- Invalid Ages (>120 or <=0) ---")
35 invalid_age = df[
36     (df["Age"] <= 0) |
37     (df["Age"] > 120)
38 ]
39 print(
40     invalid_age[
41         ["PatientID", "Name", "Age"]
42     ]
43 )
44 # Missing ages
45 print("\n--- Missing Ages ---")
46 print(
47     df[df["Age"].isnull()][
48         ["PatientID", "Name", "Age"]
49     ]
50 )
```

```

51 # Missing treatment costs
52 print("\n--- Missing Treatment Costs ---")
53 print(
54     df[df["TreatmentCost"].isnull()][
55         ["PatientID", "Name", "TreatmentCost"]
56     ]
57 )
58
59 # Convert dates temporarily for validation
60
61 admission_temp = pd.to_datetime(
62     df["AdmissionDate"],
63     dayfirst=True,
64     errors="coerce"
65 )
66 discharge_temp = pd.to_datetime(
67     df["DischargeDate"],
68     dayfirst=True,
69     errors="coerce"
70 )
71 # Invalid dates
72 print("\n--- Invalid Date Values ---")
73 print(
74     df[
75         admission_temp.isnull() |
76         discharge_temp.isnull()
77     ][
78         ["PatientID", "AdmissionDate", "DischargeDate"]]
79
80 # Discharge before admission
81 print("\n--- Discharge Before Admission ---")
82 invalid_stay = df[
83     discharge_temp < admission_temp
84 ]
85 print(
86     invalid_stay[
87         [
88             "PatientID",
89             "AdmissionDate",
90             "DischargeDate"
91         ]
92     ]
93 )
94 # Inconsistent diagnosis
95 print("\n--- Original Diagnosis Values ---")
96 print(df["Diagnosis"].unique())

```

```

97 # 3. BEFORE CLEANING ANALYSIS
98 print("\n" + "=" * 70)
99 print("2. ANALYSIS BEFORE CLEANING")
100 print("=" * 70)
101 # Average treatment cost before cleaning
102 average_cost_before = df["TreatmentCost"].mean()
103 print(
104     "\nAverage Treatment Cost BEFORE cleaning:",
105     average_cost_before)
106 # Calculate stay duration before cleaning
107 df["AdmissionTemp"] = admission_temp
108 df["DischargeTemp"] = discharge_temp
109 df["StayBefore"] = (
110     df["DischargeTemp"] -
111     df["AdmissionTemp"]).dt.days
112 print("\nHospital Stay Duration BEFORE cleaning:")
113 print(
114     df[
115         ["PatientID", "StayBefore"]])
116 # Average stay including invalid negative duration
117 average_stay_before = df["StayBefore"].mean()
118 print(
119     "\nAverage Hospital Stay BEFORE cleaning:",
120     average_stay_before,
121     "days")
122
123 # 4. CLEANING
124
125 cleaned_df = df.copy()
126 print("\n" + "=" * 70)
127 print("3. DATA CLEANING")
128 print("=" * 70)
129
130 # A. Handle missing PatientID
131 # In a real hospital system, recover from registration database.
132 # For this assignment, assign P1004.
133 cleaned_df.loc[
134     cleaned_df["PatientID"].isnull(),
135     "PatientID"
136 ] = "P1004"
137 # B. Remove duplicate records
138 cleaned_df = cleaned_df.drop_duplicates()
139 # C. Handle invalid age
140 cleaned_df.loc[
141     (cleaned_df["Age"] <= 0) |
142     (cleaned_df["Age"] > 120),
143     "Age"
144 ] = None
145 # D. Handle missing age
146 # Use median age for this assignment
147 median_age = cleaned_df["Age"].median()
148 cleaned_df["Age"] = cleaned_df["Age"].fillna(
149     median_age
150 )

```

```

151 # E. Standardize Diagnosis
152 # Convert to lowercase and remove spaces
153 cleaned_df["Diagnosis"] = (
154     cleaned_df["Diagnosis"]
155     .str.strip()
156     .str.lower()
157 )
158 # Diagnosis mapping
159 diagnosis_mapping = {
160     "diabetes": "Diabetes",
161     "diabtes": "Diabetes",
162     "hypertension": "Hypertension",
163     "asthma": "Asthma"
164 }
165 cleaned_df["Diagnosis"] = (
166     cleaned_df["Diagnosis"]
167     .map(diagnosis_mapping)
168 )
169 # F. Standardize dates
170 cleaned_df["AdmissionDate"] = pd.to_datetime(
171     cleaned_df["AdmissionDate"],
172     dayfirst=True,
173     errors="coerce"
174 )
175 cleaned_df["DischargeDate"] = pd.to_datetime(
176     cleaned_df["DischargeDate"],
177     dayfirst=True,
178     errors="coerce"
179 )

```

```

180 # G. Detect invalid discharge dates
181 cleaned_df.loc[
182     cleaned_df["DischargeDate"] <
183     cleaned_df["AdmissionDate"],
184     "DischargeDate"
185 ] = pd.NaT
186 # H. Handle missing treatment cost
187 # Fill missing treatment cost using
188 # the median cost for the corresponding diagnosis.
189 diagnosis_median = (
190     cleaned_df
191     .groupby("Diagnosis")["TreatmentCost"]
192     .transform("median")
193 )
194 cleaned_df["TreatmentCost"] = (
195     cleaned_df["TreatmentCost"]
196     .fillna(diagnosis_median)
197 )
198

```

```

218 # 6. DISPLAY CLEANED DATA
219 print("\n" + "=" * 70)
220 print("CLEANED DATASET")
221 print("=" * 70)
222 print(
223     cleaned_df[
224         ["PatientID", "Name", "Age", "Gender", "AdmissionDate", "DischargeDate", "Diagnosis", "TreatmentCost", "DoctorID", "StayDuration"]]
225 )
226

```

```

227 # 7. AFTER CLEANING ANALYSIS
228 print("\n" + "=" * 70)
229 print("5. ANALYTICAL COMPARISON")
230 print("=" * 70)
231 # Average treatment cost AFTER cleaning
232 average_cost_after = (
233     cleaned_df["TreatmentCost"].mean()
234 )
235 # Average stay AFTER cleaning
236 average_stay_after = (
237     cleaned_df["StayDuration"].mean()
238 )
239 print(
240     "\nAverage Treatment Cost BEFORE cleaning:",
241     round(average_cost_before, 2)
242 )
243 print(
244     "Average Treatment Cost AFTER cleaning:",
245     round(average_cost_after, 2)
246 )
247 print(
248     "\nAverage Hospital Stay BEFORE cleaning:",
249     round(average_stay_before, 2),
250     "days"
251 )

```

```

252 print(
253     "Average Hospital Stay AFTER cleaning:",
254     round(average_stay_after, 2),
255     "days"
256 )
257 # Difference
258 cost_difference = (
259     average_cost_after -
260     average_cost_before
261 )
262 stay_difference = (
263     average_stay_after -
264     average_stay_before
265 )
266 print(
267     "\nChange in Average Treatment Cost:",
268     round(cost_difference, 2)
269 )
270 print(
271     "Change in Average Hospital Stay:",
272     round(stay_difference, 2),
273     "days"
274 )

```

```

276 # 8. DIAGNOSIS-WISE ANALYSIS
277 print("\n" + "=" * 70)
278 print("DIAGNOSIS-WISE ANALYSIS")
279 print("=" * 70)
280 diagnosis_analysis = (
281     cleaned_df
282     .groupby("Diagnosis")
283     .agg(
284         PatientCount=("PatientID", "count"),
285         AverageCost=("TreatmentCost", "mean"),
286         AverageStay=("StayDuration", "mean")
287     )
288     .round(2)
289 )
290 print(diagnosis_analysis)
291

```

```

292 # 9. DOCTOR-WISE ANALYSIS
293 print("\n" + "=" * 70)
294 print("DOCTOR-WISE ANALYSIS")
295 print("=" * 70)
296 doctor_analysis = (
297     cleaned_df
298     .groupby("DoctorID")
299     .agg(
300         PatientCount=("PatientID", "count"),
301         AverageCost=("TreatmentCost", "mean"),
302         AverageStay=("StayDuration", "mean")
303     )
304     .round(2)
305 )
306 print(doctor_analysis)
307

```

```

308 # 10. DATA GOVERNANCE REPORT
309 print("\n" + "=" * 70)
310 print("6. DATA GOVERNANCE RECOMMENDATIONS")
311 print("=" * 70)
312 governance = [
313     "1. Make PatientID mandatory and unique.",
314     "2. Apply validation rules for age and treatment cost.",
315     "3. Ensure discharge date is not before admission date.",
316     "4. Use a controlled vocabulary for diagnosis names.",
317     "5. Store all dates in YYYY-MM-DD format.",
318     "6. Detect duplicate patient/encounter records.",
319     "7. Maintain audit logs for data modifications.",
320     "8. Perform regular data-quality audits.",
321     "9. Train staff on standardized data-entry procedures.",
322     "10. Use database constraints such as NOT NULL, UNIQUE and CHECK."
323 ]
324 for item in governance:
325     print(item)

```